

Steam Hidden Gem Score

Instructivo Completo de Replicacion

Equipo 3 | Ingenieria de Datos | UNAM 2026

Descripcion del proyecto

Pipeline de datos end-to-end en AWS que extrae informacion publica de Steam via su Web API, la almacena y procesa en capas Bronze-Silver-Gold, calcula un Hidden Gem Score (HGS) para identificar videojuegos de alta calidad con poca visibilidad, y carga los resultados en Amazon RDS PostgreSQL.

Servicios AWS utilizados:

- Amazon S3 (almacenamiento por capas Bronze, Silver, Gold)
- AWS Glue (ETL + Data Quality con 13 reglas)
- Amazon EC2 (extraccion Bronze y calculo Gold Layer)
- Amazon RDS PostgreSQL (Gold Layer — tablas hidden_gems y genre_summary)
- Amazon EventBridge Scheduler (orquestracion diaria)
- Amazon Athena (analisis SQL ad-hoc sobre Silver Layer)
- AWS IAM (roles y permisos por servicio)

Formula Hidden Gem Score:

$HGS = 0.50 \times \text{Calidad} + 0.30 \times \text{Obscuridad} + 0.20 \times \text{Precio}$

1. Crear y configurar la cuenta AWS

1.1 Crear la cuenta

1. Ir a <https://aws.amazon.com> > clic en Crear una cuenta de AWS
2. Ingresar correo electronico, nombre de cuenta y contrasena segura
3. Tipo de cuenta: Personal (uso academico)
4. Ingresar datos de tarjeta de credito (necesaria para verificacion; el Free Tier no genera cargos si se respetan los limites)
5. Verificar identidad por telefono
6. Seleccionar plan Support: Basic (gratis)
7. Esperar correo de confirmacion y acceder a <https://console.aws.amazon.com>

⚠ *Nota: Todos los servicios de este proyecto caben en el AWS Free Tier si se usan moderadamente. Revisar el Cost Explorer periodicamente para evitar cargos inesperados.*

1.2 Activar MFA en la cuenta root

8. Clic en nombre de cuenta (arriba derecha) > Security credentials
9. Multi-factor authentication > Assign MFA device
10. Elegir Authenticator app, escanear el QR con Google Authenticator o Authy
11. Ingresar dos codigos consecutivos para confirmar

✓ Siempre usar MFA en la cuenta root. Para trabajo diario, crear usuarios IAM con permisos especificos.

1.3 Seleccionar la region

12. En la consola AWS, clic en el selector de region (esquina superior derecha)
13. Seleccionar: US East (Ohio) us-east-2

⚠ *Nota: Verificar que el selector muestre us-east-2 ANTES de crear cualquier recurso. Todos los servicios del proyecto deben estar en la misma region.*

2. Configurar roles IAM

IAM controla los permisos de cada servicio. El proyecto requiere los siguientes roles:

Rol IAM	Trusted entity	Para que se usa
EC2S3AccessRole	ec2	Permite a EC2 leer/escribir en S3 sin credenciales hardcoded
TEST-GLUE-ROLE	glue	Permite a Glue leer Bronze, escribir Silver y registrar logs
rds-monitoring-role	monitoring.rds	Monitoreo interno de RDS (AWS lo crea automaticamente)

Rol IAM	Trusted entity	Para que se usa
AWSServiceRoleForRDS	rds (Service-Linked)	Rol de servicio de RDS, creado por AWS automáticamente
steamApi-role-*	lambda	Roles de Lambda para consultas auxiliares a Steam API

Role name	Trusted entities
AWSServiceRoleForRDS	AWS Service: rds (Service-Linked Role)
AWSServiceRoleForResourceExplorer	AWS Service: resource-explorer-2 (Service-Linked Role)
AWSServiceRoleForSupport	AWS Service: support (Service-Linked Role)
AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service-Linked Role)
EC2S3AccessRole	AWS Service: ec2
rds-monitoring-role	AWS Service: monitoring.rds
steamApi-role-b7zvm0nb	AWS Service: lambda
steamApi-role-qgjy5uc6	AWS Service: lambda
TEST-GLUE-ROLE	AWS Service: glue

Figura: Roles IAM configurados en la cuenta AWS del proyecto

2.1 Crear EC2S3AccessRole

14. AWS Console > IAM > Roles > Create role
15. Trusted entity: AWS service | Use case: EC2
16. Agregar política: AmazonS3FullAccess
17. Nombre: EC2S3AccessRole > Create role

2.2 Crear TEST-GLUE-ROLE

18. AWS Console > IAM > Roles > Create role
19. Trusted entity: AWS service | Use case: Glue
20. Agregar políticas: AmazonS3FullAccess + AWSGlueServiceRole + CloudWatchLogsFullAccess
21. Nombre: TEST-GLUE-ROLE > Create role

*⚠ Nota: Los roles **AWSServiceRoleForRDS** y **rds-monitoring-role** los crea AWS automáticamente al crear la instancia RDS.*

3. Crear el bucket S3 y la estructura de carpetas

3.1 Bucket principal

22. AWS Console > S3 > Create bucket
23. Nombre: unam-2026-ingenieriadedatos-equipo3-<account-id>-us-east-2-an

24. Region: US East (Ohio) us-east-2
25. Block Public Access: activado (todas las opciones marcadas)
26. Encryption: SSE-S3 (por defecto)
27. Hacer clic en Create bucket

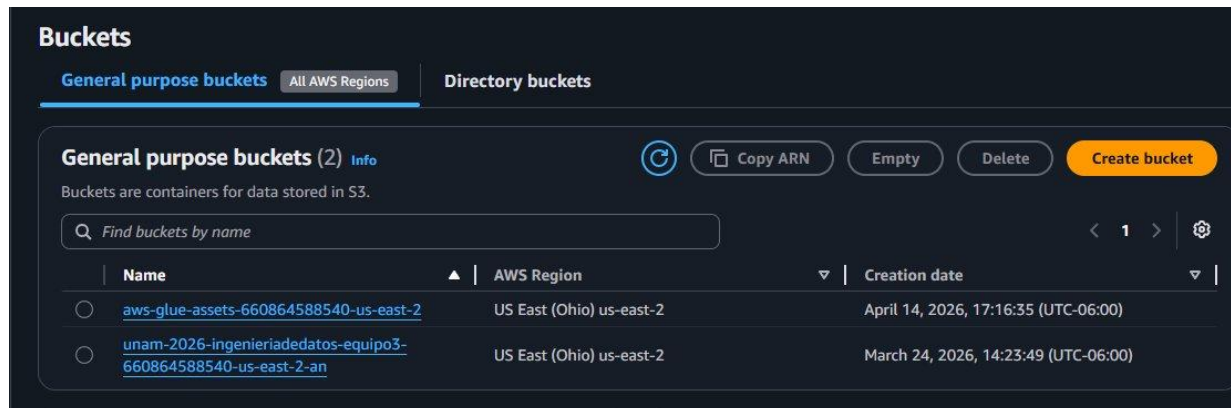


Figura: Los dos buckets S3 del proyecto: bucket principal y bucket de assets de Glue (creado automaticamente)

3.2 Crear las 5 carpetas dentro del bucket

Prefijo	Capa	Contenido
1bronze/	Bronze Layer	Datos crudos JSONL extraidos de Steam API, particionados por fecha
2silver/	Silver Layer	Datos limpios y validados en formato Parquet (.snappy.parquet)
3gold/	Gold Layer	Respaldo de resultados finales con HGS calculado
athena/	Athena results	Resultados de consultas SQL ejecutadas en Amazon Athena
aws/	Glue / config	Scripts ETL de Glue y archivos de configuracion AWS

28. Entrar al bucket principal > Create folder > Nombre: 1bronze > Create folder
29. Repetir para: 2silver, 3gold, athena, aws

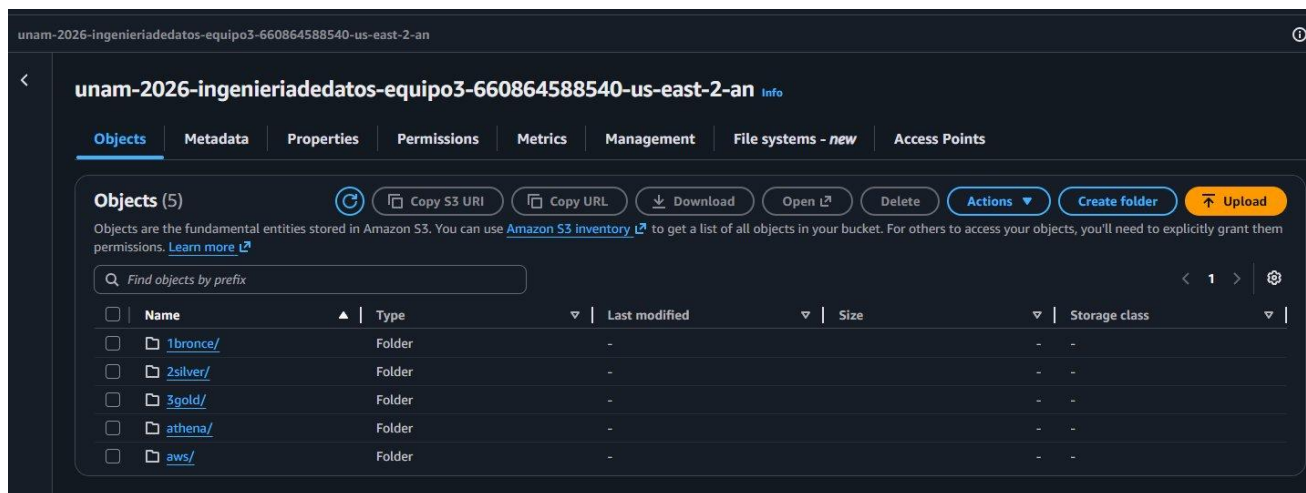


Figura: Estructura de carpetas del bucket principal con las 5 capas del pipeline

4. Configurar las instancias EC2

El proyecto usa dos instancias EC2: una principal para el pipeline de extraccion y otra (CapaGolp) dedicada a la carga en la Gold Layer.

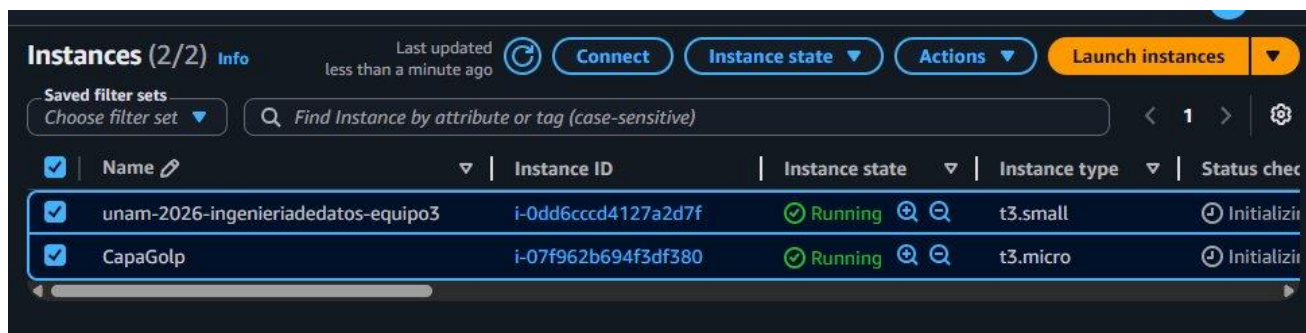


Figura: Instancias EC2 del proyecto: unam-2026-ingenieriadedatos-equipos3 (t3.small) y CapaGolp (t3.micro), ambas en estado Running

4.1 Lanzar instancia principal (pipeline de extraccion)

30. AWS Console > EC2 > Launch Instance
31. Name: unam-2026-ingenieriadedatos-equipos3
32. AMI: Ubuntu Server 22.04 LTS
33. Instance type: t3.small
34. Key pair: Create new key pair > Nombre: steam-key > RSA > .pem > Download
35. Network: Allow SSH (22) from My IP
36. IAM instance profile: EC2S3AccessRole
37. Storage: 20 GiB gp3

38. Launch Instance

4.2 Lanzar instancia Gold Layer (CapaGolp)

39. Repetir el proceso anterior con:

40. Name: CapaGolp | Instance type: t3.micro | Misma key pair steam-key

41. IAM instance profile: EC2S3AccessRole

4.3 Instalar dependencias en cada instancia

```
ssh -i steam-key.pem ubuntu@<ip-instancia>  
sudo apt update && sudo apt install python3-pip python3-venv -y  
python3 -m venv venv && source venv/bin/activate  
pip install boto3 pandas psycpg2-binary pyarrow requests
```

5. Crear la base de datos RDS PostgreSQL

5.1 Crear la instancia

42. AWS Console > Aurora and RDS > Crear base de datos

43. Metodo: Configuracion completa (NO expressa — no permite elegir PostgreSQL)

44. Motor: PostgreSQL | Version: 15.x

45. Templates: Free tier

46. DB instance identifier: unamdatabaseingdedatoequipo3

47. Master username: dbadmin | Password: definir una contraseña segura

48. DB instance class: db.t4g.micro (Free Tier)

49. Storage: 20 GiB | Storage autoscaling: desactivar

50. VPC: la misma que las instancias EC2 | Public access: No

51. VPC security group: crear nuevo | Database name: steam_gold

52. Hacer clic en Create database

The screenshot displays the AWS Management Console for an Amazon RDS instance. The instance is named 'unamdatabaseingdedatoequipo3' and is in the 'Available' state. It is a PostgreSQL engine instance with the class 'db.t4g.micro' and is located in the 'us-east-2c' region. The CPU usage is shown as 3.87%. The console also shows the 'Connectivity & security' tab, which includes options for connecting using code snippets, CloudShell, or endpoints. The 'Internet access gateway' is disabled, and 'IAM Authentication' is also disabled. The 'Programming language' is set to 'PSQL (macOS)' and the 'Endpoint type' is 'Instance endpoint'. The 'Connection steps' section provides a list of commands to connect to the instance.

Summary

DB identifier unamdatabaseingdedatoequipo3	Status Available	Role Instance	Engine PostgreSQL
CPU 3.87%	Class db.t4g.micro	Current activity 0.00 sessions	Region & AZ us-east-2c

Recommendations
1 Informational

Connect using | Info

- ☒ **Code snippets**
Use when connecting through SDK, APIs, or third-party tools including agents.
- ☐ **CloudShell**
Use for a quick access to AWS CLI that launches directly from the AWS Management Console.
- ☐ **Endpoints**
Use when connecting through any IDE interface.

Internet access gateway
☐ Disabled

IAM Authentication
☐ Disabled

Programming language
PSQL (macOS)

Endpoint type
Instance endpoint

Connection steps
Follow the steps below to paste the code of each step in your tool and run the commands. The snippets dynamically reflect the authentication configuration.

```
1 curl -o global-bundle.pem https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem
```

```
1 export RDSHOST="unamdatabaseingdedatoequipo3.cpswcmwoolha.us-east-2.rds.amazonaws.com"
```

```
2 psql "host=$RDSHOST port=5432 dbname=postgres user=dbadmin sslmode=verify-full sslrootcert=./global-bundle.pem"
```

Figura: Instancia RDS unamdatabaseingdedatoequipo3 — estado Available, motor PostgreSQL, clase db.t4g.micro, region us-east-2c

5.2 Configurar Security Group

53. EC2 > Security Groups > seleccionar el SG del RDS
54. Inbound rules > Add rule: Type: PostgreSQL | Port: 5432 | Source: SG de la EC2
55. Save rules

5.3 Conectarse y verificar desde EC2

```
# Descargar certificado SSL
curl -o global-bundle.pem
https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem

# Conectarse al RDS
psql "host=unamdatabaseingdedatoequipo3.cpswcmwoolha.us-east-2.rds.amazonaws.com port=5432 dbname=steam_gold user=dbadmin sslmode=verify-full sslrootcert=./global-bundle.pem"
```



```
(venv) ubuntu@ip-172-31-47-107:~$ psql "host=unamdatabaseingdedatoequipo3.cpswcmwoolha.us-east-2.rds.amazonaws.com port=5432 dbname=steam_gold user=dbadmin sslmode=verify-full sslrootcert=./global-bundle.pem"
Password for user dbadmin:
psql (18.3 (Ubuntu 18.3-1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

steam_gold=> \dt
          List of tables
Schema | Name          | Type  | Owner
-----+-----+-----+-----
public | genre_summary | table | dbadmin
public | hidden_gems   | table | dbadmin
(2 rows)

steam_gold=> █
```

Figura: Conexión exitosa al RDS desde EC2 — tablas `hidden_gems` y `genre_summary` creadas automáticamente por `gold_layer.py`

6. Script de extracción - Bronze Layer

6.1 Logica

`extraccion.py` consume la Steam Web API pública, obtiene lista de juegos y sus detalles, y guarda los datos crudos como JSONL en la carpeta `1bronze/` particionados por fecha.

6.2 Crear el script en EC2

```
cat > ~/extraccion.py << 'EOF'
import boto3, requests, json, datetime, time

BUCKET = 'unam-2026-ingenieriadedatos-equipo3-660864588540-us-east-2-an'
PREFIX = '1bronze/'

def get_app_list():
    url = 'https://api.steampowered.com/ISteamApps/GetAppList/v2/'
    return requests.get(url, timeout=15).json()['applist']['apps']

def get_app_details(appid):
    url =
f'https://store.steampowered.com/api/appdetails?appids={appid}&cc=us&l=en'
    try:
        r = requests.get(url, timeout=10)
        data = r.json().get(str(appid), {})
        return data.get('data') if data.get('success') else None
    except Exception:
        return None

def main():
    s3 = boto3.client('s3')
    apps = get_app_list()
    fecha = datetime.date.today().isoformat()
    key = f'{PREFIX}{fecha}/games.jsonl'
    lines = []
    for app in apps:
        detail = get_app_details(app['appid'])
```



```

        if detail:
            lines.append(json.dumps({
                'appid': app['appid'],
                'nombre': detail.get('name'),
                'precio':
detail.get('price_overview', {}).get('final', 0) / 100,
                'generos': [g['description'] for g in
detail.get('genres', [])],
                'puntuacion':
detail.get('metacritic', {}).get('score', 0),
                'metacritic_score':
detail.get('metacritic', {}).get('score', 0),
                'total_resenas':
detail.get('recommendations', {}).get('total', 0),
                'resenas_positivas':
detail.get('ratings', {}).get('positive', 0),
                'resenas_negativas':
detail.get('ratings', {}).get('negative', 0),
                'jugadores_actuales_api': 0,
            }))
            time.sleep(0.5)
            s3.put_object(Bucket=BUCKET, Key=key, Body='\n'.join(lines))
            print(f'Guardados {len(lines)} juegos en s3://{BUCKET}/{key}')

if __name__ == '__main__': main()
EOF

```

6.3 Ejecutar y verificar

```
source ~/venv/bin/activate && python ~/extraccion.py
```

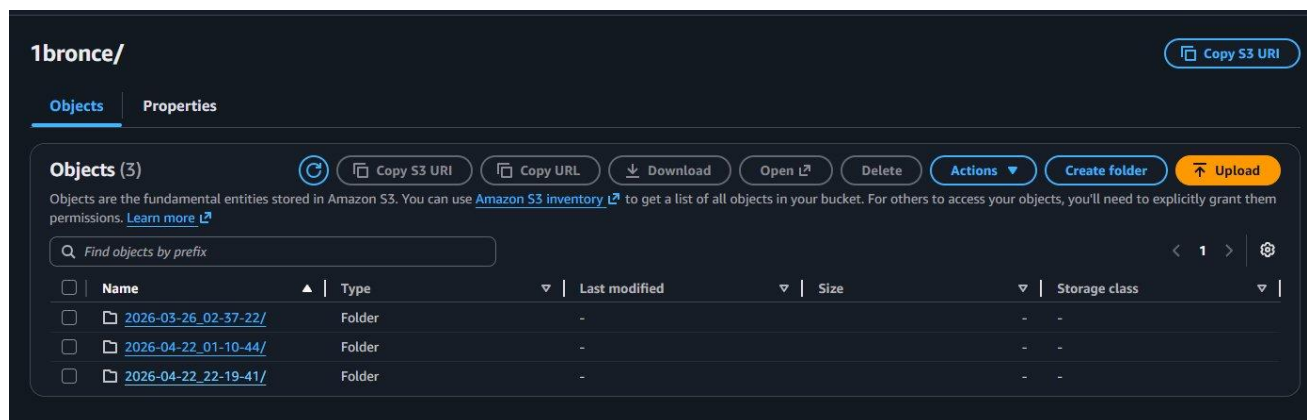


Figura: Carpeta 1bronze/ con 3 particiones de fechas distintas, cada una con los archivos JSONL de la extraccion diaria

7. Configurar AWS Glue - Bronze a Silver

7.1 Crear el Glue Job

56. AWS Console > Glue > ETL jobs > Visual ETL (boton +)

57. Nombre: TEST-GLUE-JOB-DATA-QUALITY

58. Job details: IAM Role: TEST-GLUE-ROLE | Glue version: 5.0 | Worker type: G.1X | Workers: 10

59. Guardar antes de agregar nodos

7.2 Construir el flujo visual (5 nodos en secuencia)

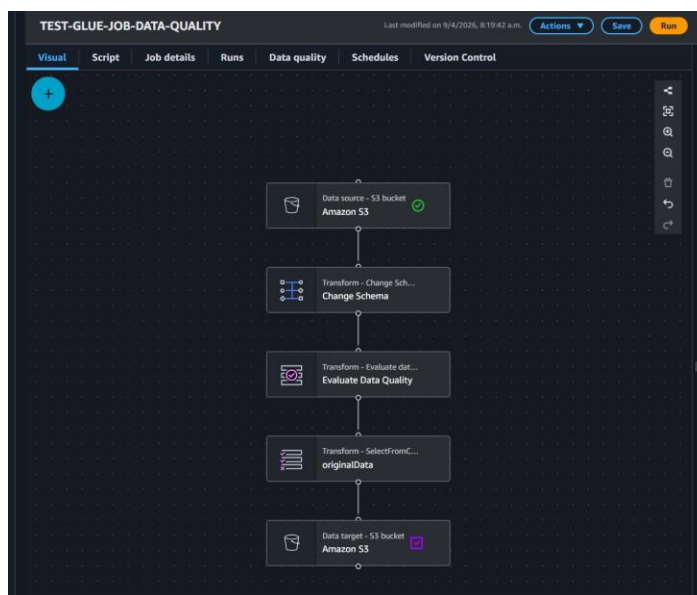


Figura: Flujo visual del Glue Job: S3 Bronze > Change Schema > Evaluate Data Quality > SelectFromCollection > S3 Silver

#	Nodo Glue	Configuracion
1	Data source - Amazon S3	Source path: s3://unam-2026.../1bronze/ Format: JSON
2	Change Schema	Renombrar y castear: appid (long), nombre (string), precio (double), puntuacion (double), metacritic_score (int), total_resenas (long), resenas_positivas (long), resenas_negativas (long), jugadores_actuales_api (long)
3	Evaluate Data Quality	13 reglas (ver seccion 7.3) Action on failure: Continue Output: originalData
4	SelectFromCollection	Nombre: originalData Filtra solo los registros que pasaron las reglas
5	Data target - Amazon S3	Target: s3://unam-2026.../2silver/ Format: Parquet Compression: Snappy

7.3 Reglas de Data Quality

En el nodo Evaluate Data Quality, pegar exactamente el siguiente bloque en el campo Rules:

```
Rules = [
  IsComplete "nombre",
  IsComplete "appid",
  IsComplete "precio",
  IsComplete "total_resenas",
  IsComplete "resenas_positivas",
  IsComplete "resenas_negativas",
  IsComplete "puntuacion",
  ColumnValues "metacritic_score" between 0 and 100,
  ColumnValues "puntuacion" between 0 and 100,
  ColumnValues "jugadores_actuales_api" >= 0,
  ColumnValues "resenas_positivas" >= 0,
  ColumnValues "resenas_negativas" >= 0,
  ColumnValues "total_resenas" >= 10
]
```

#	Regla	Que valida
1	IsComplete "nombre"	nombre del juego no puede ser nulo
2	IsComplete "appid"	identificador unico de Steam requerido
3	IsComplete "precio"	precio no puede ser nulo (F2P = 0)
4	IsComplete "total_resenas"	total de resenas requerido
5	IsComplete "resenas_positivas"	resenas positivas requeridas
6	IsComplete "resenas_negativas"	resenas negativas requeridas
7	IsComplete "puntuacion"	puntuacion de Steam requerida
8	ColumnValues "metacritic_score" between 0 and 100	score Metacritic en rango valido
9	ColumnValues "puntuacion" between 0 and 100	puntuacion Steam en rango 0-100
10	ColumnValues "jugadores_actuales_api" >= 0	jugadores actuales no puede ser negativo
11	ColumnValues "resenas_positivas" >= 0	resenas positivas >= 0
12	ColumnValues "resenas_negativas" >= 0	resenas negativas >= 0
13	ColumnValues "total_resenas" >= 10	minimo 10 resenas para incluir el juego

7.4 Ejecutar y verificar

60. Save > Run

61. Pestana Runs: esperar estado Succeeded (1-2 minutos con 10 workers G.1X)

62. Verificar archivos .snappy.parquet en s3://.../2silver/

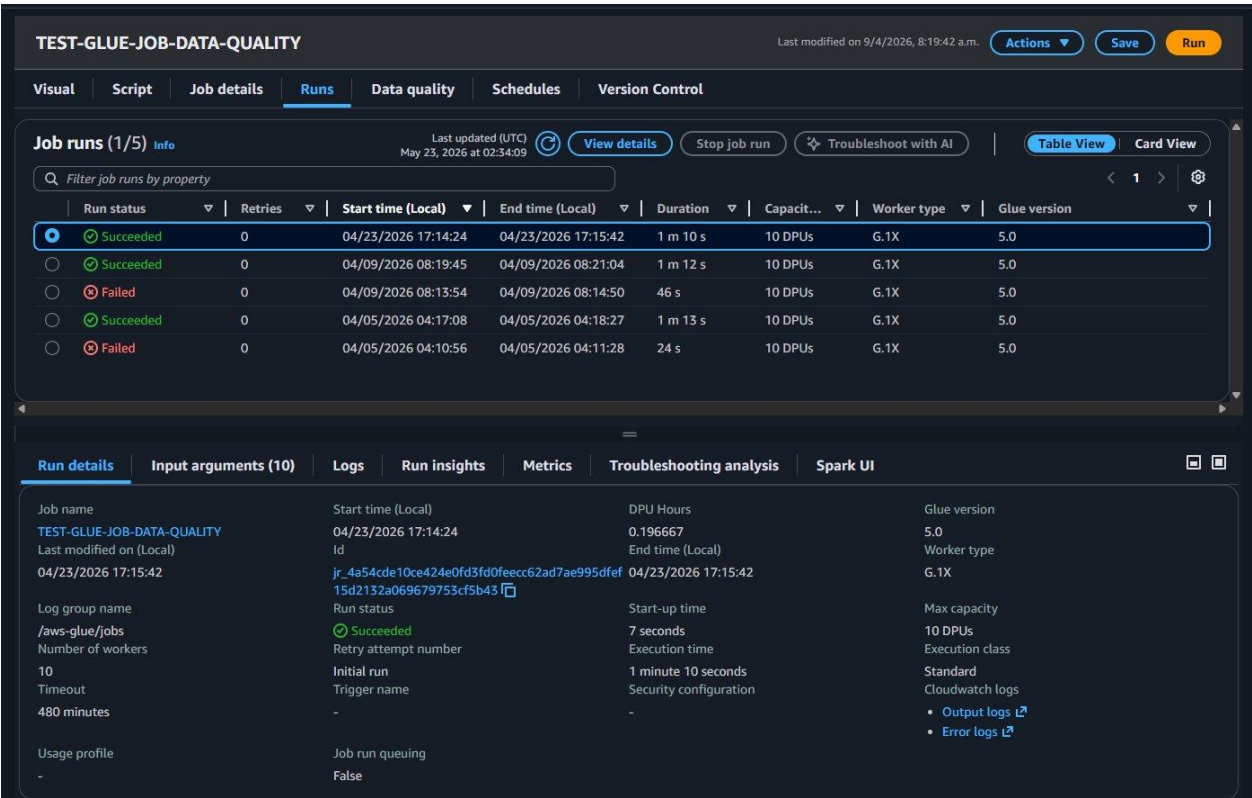


Figura: Historial de ejecuciones del Glue Job — 3 runs Succeeded y 2 Failed (los primeros intentos durante configuracion inicial)

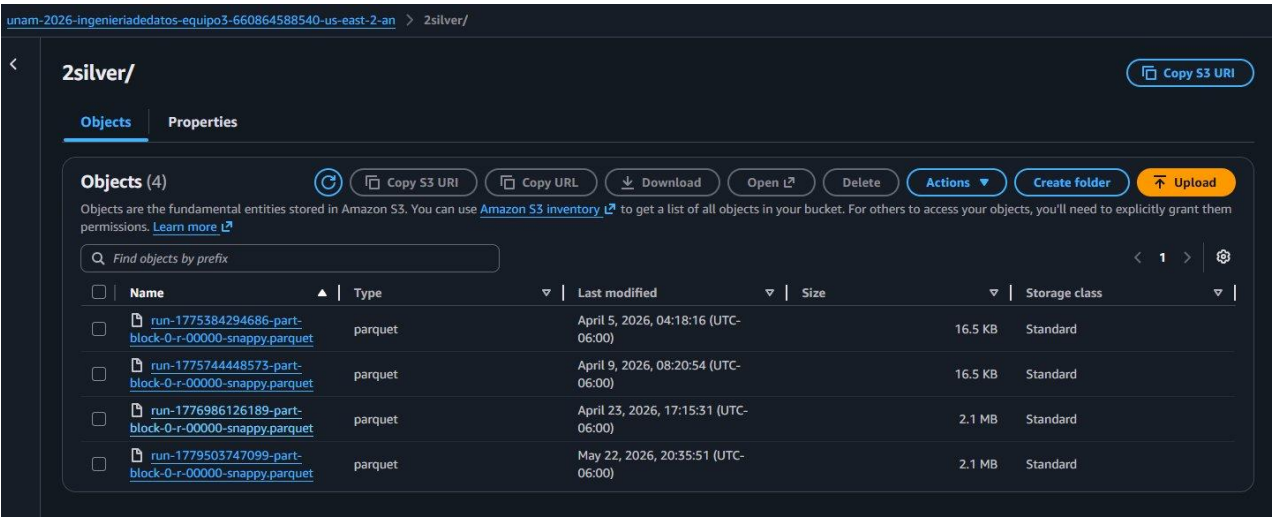


Figura: Carpeta 2silver/ con 4 archivos Parquet comprimidos con Snappy, generados por cada ejecucion exitosa del Glue Job

8. Script Gold Layer - Calculo HGS y carga en RDS

8.1 Variables de entorno

Variable	Valor ejemplo	Descripción
S3_BUCKET	unam-2026-ingenieriadedatos-equipos3-...-us-east-2-an	Nombre exacto del bucket principal
S3_PREFIX	2silver/	Prefijo Silver Layer en S3
DB_HOST	unamdatabaseingdedatoequipo3.cpswcmwoolha.us-east-2.rds.amazonaws.com	Endpoint del RDS
DB_NAME	steam_gold	Base de datos creada en RDS
DB_USER	dbadmin	Usuario maestro de RDS
DB_PASSWORD	tu-contrasena-segura	Contraseña definida al crear el RDS

8.2 Configurar y ejecutar en la instancia CapaGolp

```
# 1. Exportar variables
export S3_BUCKET='unam-2026-ingenieriadedatos-equipos3-660864588540-us-east-2-an'
export S3_PREFIX='2silver/'
export DB_HOST='unamdatabaseingdedatoequipo3.cpswcmwoolha.us-east-2.rds.amazonaws.com'
export DB_NAME='steam_gold'
export DB_USER='dbadmin'
export DB_PASSWORD='tu-contrasena'

# 2. Ejecutar
source ~/venv/bin/activate
python ~/gold_layer.py
```

8.3 Formula HGS aplicada

- $quality_score = \text{resenas_positivas} / \text{total_resenas}$
- $obscurity_score = 1 - (\text{total_resenas} / \text{max_total_resenas_del_dataset})$
- $price_score = 1.0$ si precio = 0 (F2P), sino $\max(0, 1 - \text{precio} / 60)$
- $HGS = 0.50 \times quality + 0.30 \times obscurity + 0.20 \times price$
- Tier S si $HGS \geq 0.85$ | Tier A si $HGS \geq 0.70$ | Tier B resto

8.4 Tablas resultantes en RDS

- **hidden_gems:** appid, nombre, generos, precio, puntuacion, total_resenas, resenas_positivas, hidden_gem_score, tier, updated_at

- genre_summary: genero, total_juegos, avg_hgs, top_appid
-

9. Amazon Athena — analisis SQL ad-hoc

9.1 Configurar resultados

63. AWS Console > Amazon Athena > Settings > Manage
64. Query result location: s3://unam-2026-.../athena/
65. Save

9.2 Crear tabla externa sobre Silver Layer

```
CREATE EXTERNAL TABLE IF NOT EXISTS steam_silver (  
  appid          BIGINT,  
  nombre         STRING,  
  precio         DOUBLE,  
  generos        ARRAY<STRING>,  
  puntuacion     DOUBLE,  
  metacritic_score INT,  
  total_resenas  BIGINT,  
  resenas_positivas BIGINT,  
  resenas_negativas BIGINT,  
  jugadores_actuales_api BIGINT  
)  
STORED AS PARQUET  
LOCATION 's3://unam-2026-ingenieriadedatos-equipo3-660864588540-us-east-2-an/2silver/'  
TBLPROPERTIES ('parquet.compression'='SNAPPY');
```

9.3 Consultas de ejemplo

```
-- Top 10 juegos por puntuacion  
SELECT nombre, precio, puntuacion, total_resenas  
FROM steam_silver WHERE total_resenas >= 10  
ORDER BY puntuacion DESC LIMIT 10;
```

10. Automatización con EventBridge Scheduler

10.1 Cron en EC2

```
crontab -e  
# Agregar (ejecucion diaria 3am UTC):  
0 3 * * * cd /home/ubuntu && source venv/bin/activate && python extraccion.py  
&& python gold_layer.py >> pipeline.log 2>&1
```

10.2 Regla en EventBridge

66. AWS Console > EventBridge > Scheduler > Create schedule

67. Nombre: steam-pipeline-daily

68. Cron: 0 3 1 1,5,9 ? * (Esto significa: el día 1 de enero, mayo y septiembre a las 3:00am UTC — que es cada 4 meses.)

69. Timezone: seleccionar America/Mexico_City si quieres que corra a las 3am hora local.

70. Flexible time window: Off

71. Target: seleccionar el tipo de target según tu implementación:

Si se usa EC2 Run Command → AWS SDK: SSM > SendCommand

Si se usa Lambda → AWS Lambda > Invoke

72. Action after schedule completion: NONE

73. Permissions: crear un nuevo rol o usar uno existente con permisos para invocar el target

74. Clic en Create schedule

Specify schedule detail

Schedule name and description

Schedule name
steam-pipeline
Use only letters, numbers, dashes, dots or underscores. Max 64 characters.

Description - optional
Ejecuta el pipeline de Steam cada 4 meses
Maximum of 512 characters.

Schedule group
Each schedule needs to be placed in a schedule group. By default, a schedule is placed in the "Default" group. You can also [create your own schedule group](#). You can only add tags to a schedule group, not a schedule.
default

Schedule pattern

Occurrence [Info](#)
You can define an one-time or recurrent schedule.
☐ One-time schedule ☒ Recurring schedule

Time zone
The time zone for the schedule.
(UTC-06:00) America/Mexico_City

Schedule type
Choose the schedule type that best meets your needs.
☒ Cron-based schedule
A schedule set using a cron expression that runs at a specific time, such as 8:00 a.m. PST on the first Monday of every month.
☐ Rate-based schedule
A schedule that runs at a regular rate, such as every 10 minutes.

Cron expression [Info](#)
Define the cron expression for the schedule
Copy Clear
cron (0 3 1 1,5,9 ? *)
Minutes Hours Day of month Month Day of the week Year

Next 10 trigger dates
Date and time are displayed in your current time zone in UTC format, e.g. "Wed, Nov 9, 2022 09:00 (UTC - 08:00)" for Pacific time
Tue, 01 Sep 2026 03:00:00 (UTC-06:00)
Fri, 01 Jan 2027 03:00:00 (UTC-06:00)
Sat, 01 May 2027 03:00:00 (UTC-06:00)
Wed, 01 Sep 2027 03:00:00 (UTC-06:00)
Sat, 01 Jan 2028 03:00:00 (UTC-06:00)
Mon, 01 May 2028 03:00:00 (UTC-06:00)
Fri, 01 Sep 2028 03:00:00 (UTC-06:00)
Mon, 01 Jan 2029 03:00:00 (UTC-06:00)
Tue, 01 May 2029 03:00:00 (UTC-06:00)
Sat, 01 Sep 2029 03:00:00 (UTC-06:00)

⚠ Nota: La expresion cron en EventBridge usa 6 campos: minutos horas dia-mes mes dia-semana ano. El ? significa cualquier valor.

Para verificar que la expresión es correcta, en el mismo formulario aparece un preview con las próximas fechas de ejecución — debe mostrar algo como:

Next 10 trigger dates
Date and time are displayed in your current time zone in UTC format, e.g. "Wed, Nov 9, 2022 09:00 (UTC - 08:00)" for Pacific time

Tue, 01 Sep 2026 03:00:00 (UTC-06:00)
Fri, 01 Jan 2027 03:00:00 (UTC-06:00)
Sat, 01 May 2027 03:00:00 (UTC-06:00)
Wed, 01 Sep 2027 03:00:00 (UTC-06:00)
Sat, 01 Jan 2028 03:00:00 (UTC-06:00)
Mon, 01 May 2028 03:00:00 (UTC-06:00)
Fri, 01 Sep 2028 03:00:00 (UTC-06:00)
Mon, 01 Jan 2029 03:00:00 (UTC-06:00)
Tue, 01 May 2029 03:00:00 (UTC-06:00)
Sat, 01 Sep 2029 03:00:00 (UTC-06:00)

11. Verificar el pipeline completo

11.1 Consultas de validacion en RDS

```
psql "host=unamdatabaseingdedatoequipo3.cpswcmwoolha.us-east-2.rds.amazonaws.com port=5432 dbname=steam_gold user=dbadmin sslmode=verify-full sslrootcert=./global-bundle.pem"

-- Distribucion por tier
SELECT tier, COUNT(*) AS total FROM hidden_gems GROUP BY tier ORDER BY tier;

-- Top 10 joyas ocultas
SELECT nombre, hidden_gem_score, tier FROM hidden_gems ORDER BY
hidden_gem_score DESC LIMIT 10;

-- Generos con mayor HGS promedio
SELECT genero, total_juegos, ROUND(avg_hgs::numeric,4) FROM genre_summary ORDER
BY avg_hgs DESC LIMIT 10;
```

12. Resumen del flujo completo

#	Servicio	Accion	Entrada	Salida
1	EventBridge	Dispara pipeline 4 meses	Schedule cron	Trigger EC2
2	EC2 + Python	extraccion.py - Steam API	Steam Web API	JSONL en 1bronce/
3	AWS Glue	ETL + 13 reglas DQ	1bronce/ JSONL	Parquet en 2silver/
4	EC2 + Python	gold_layer.py - calcula HGS	2silver/ Parquet	Upsert RDS
5	RDS PostgreSQL	hidden_gems + genre_summary	gold_layer.py	Consultas / API
6	Amazon Athena	Analisis SQL ad-hoc	2silver/ Parquet	athena/ results

Resultado final: la tabla hidden_gems en RDS PostgreSQL contiene el HGS de cada juego con su tier S/A/B. Los datos se actualizan diariamente de forma automatica y estan listos para ser consumidos por una API, dashboard o cualquier herramienta de visualizacion.